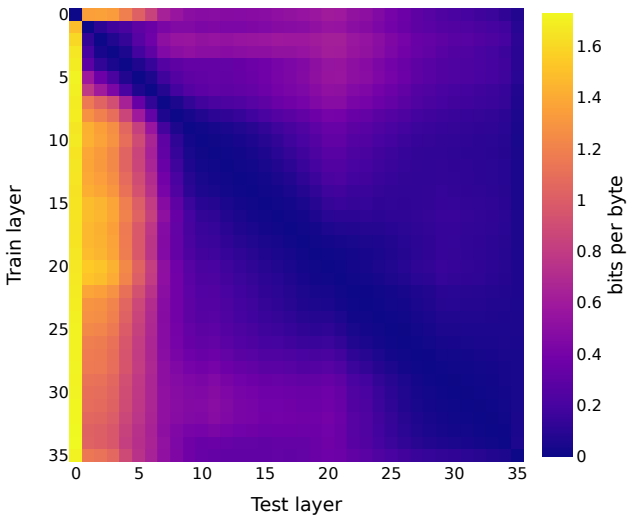

Figure 7. Transfer penalties for Pythia 12B. Each row corresponds to a single tuned lens probe trained on layer ℓ, and each column is a layer ℓ' on which probes are evaluated. Each cell shows the cross-entropy loss of probe ℓ evaluated on layer ℓ', *minus* its on-distribution loss (so that the diagonal entries are identically zero).

its affine translators from the lens trained on the base model.

As an example of the transferability of lenses to fine-tuned models we use Vicuna 13B (Chiang et al., 2023), an open source instruct fine-tuned chat model based on LLaMA 13B (Touvron et al., 2023). In Figure 13, Section A, we compare the performance of a lens specifically trained on Vicuna to a tuned lens trained only on LLaMA. Both Tuned Lenses were trained using a subsample of the RedPajama dataset (Together, 2023), an open source replication of LLaMA's training set. The performance of both lenses was then evaluated on the test set of Anthropic's Helpful Harmless (Bai et al., 2022) conversation dataset and the RedPajama Dataset. On the RedPajama Dataset we find, at worst, a 0.3 bits per byte increase in KL divergence to the models final output. On the helpful harmless corpus we find no significant difference between the transferred and trained lenses. These results show that fine-tuning minimally effects the representations used by the tuned lens. This opens applications of the tuned lens for monitoring changes in the representations of a module during fine-tuning and minimizes the need for practitioners to train lenses on new fine-tuned models.

Relation to model stitching. The tuned lens can be viewed as a way of "stitching" an intermediate layer directly onto the unembedding, with an affine transform in between to align the representations. The idea of model stitching was introduced by Lenc and Vedaldi (2015), who form a composite model out of two *frozen* pretrained models A and B, by connecting the bottom layers of A to the top layers of B. An affine transform suffices to stitch together independently trained models with minimal performance loss (Bansal et al., 2021; Csiszárik et al., 2021). The success of the tuned lens shows that model stitching works for different layers inside a single model as well.

Benefits over traditional probing. Unlike Alain and Bengio (2016), who train early exiting probes for image classifiers, we do not learn a new unembedding for each layer. This is important, since it allows us to shrink the size of each learned matrix from $|\mathcal{V}| \times d$ to $d \times d$, where $|\mathcal{V}|$ ranges from 50K (GPT-2, Pythia) to over 250K (BLOOM). We observe empirically that training a new unembedding matrix requires considerably more training steps and a larger batch size than training a translator, and often converges to a worse perplexity.

4. Measuring Causal Fidelity

Prior work has argued that interpretability hypotheses should be tested with causal experiments: an interpretation of a neural network should make predictions about what will happen when we intervene on its weights or activations (Olah et al., 2020; Chan et al., 2022). This is especially important for probing techniques, since it's known that probes can learn to rely on spurious features unrelated to the model's performance (Hewitt and Liang, 2019; Belinkov, 2022).

To explore whether the tuned lens finds causally relevant features, we will assess two desired properties:

1. Latent directions that are important to the tuned lens should also be important to the final layer output. Concretely, if the tuned lens *relies on* a feature[4] $\boldsymbol{v}$ in the residual stream (its output changes significantly when we manipulate $\boldsymbol{v}$) then the model output should also change a lot when we manipulate $\boldsymbol{v}$.

2. These latent directions should be important *in the same way* for both the tuned lens and the model. Concretely, if we manipulate the hidden state so that the tuned lens changes in a certain way (e.g. doubling the probability assigned to "dog") then the model output should change similarly. We will call this property *stimulus-response alignment*.

4.1. Causal basis extraction

To test Property 1, we first need to find the important directions for the tuned lens. Amnesic probing (Elazar et al.,

[4]For simplicity we assume the "features as directions" hypothesis (Elhage et al., 2022), which defines a "feature" to be the one-dimensional subspace spanned by a unit vector $\boldsymbol{v}$.

2021) provides one way to do this—it seeks a direction whose erasure maximally degrades a model's accuracy.

However, this only elicits a single important direction, whereas we would like to find many such directions. To do so, we borrow intuition from PCA, searching for additional directions that also degrade accuracy, but which are orthogonal to the original amnesic direction. This leads to a method that we call **causal basis extraction** (CBE), which finds the the principal features used by a model.

More specifically, let f be a function (such as the tuned lens) that maps latent vectors $\boldsymbol{h} \in \mathbb{R}^d$ to logits $\boldsymbol{y}$. Let $r(\boldsymbol{h}, \boldsymbol{v})$ be an erasure function which removes information along the span of $\boldsymbol{v}$ from $\boldsymbol{x}$. In this work we use $r(\boldsymbol{h}, \boldsymbol{v})$ is *mean ablation*, which sets $\langle r(\boldsymbol{h}, \boldsymbol{v}), \boldsymbol{v} \rangle$ to the mean value of $\langle \boldsymbol{h}, \boldsymbol{v} \rangle$ in the dataset (see Appendix D.1). We define the *influence* σ of a unit vector $\boldsymbol{v}$ to be the expected KL divergence between the outputs of f before and after erasing $\boldsymbol{v}$ from $\boldsymbol{h}$:

$$\sigma(\boldsymbol{v}; f) = \mathop{\mathbb{E}}_{\boldsymbol{h}} \Big[D_{KL}(f(\boldsymbol{h}) \,||\, f(r(\boldsymbol{h}, \boldsymbol{v}))) \Big] \tag{10}$$

We seek to find an orthonormal basis $B = (\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k)$ containing principal features of f, ordered by a sequence of influences $\Sigma = (\sigma_1, \ldots, \sigma_k)$ for some $k \leq d$. In each iteration we search for a feature $\boldsymbol{v}_i$ of maximum influence that is orthogonal to all previous features $\boldsymbol{v}_j$:

$$\begin{aligned} \boldsymbol{v}_i &= \underset{||\boldsymbol{v}||_2 = 1}{\operatorname{argmax}} \ \sigma(\boldsymbol{v}; f) \\ \text{s.t.} \quad & \langle \boldsymbol{v}, \boldsymbol{v}_j \rangle = \boldsymbol{0}, \quad \forall j < i \end{aligned} \tag{11}$$

With a perfect optimizer, the influence of $\boldsymbol{v}_i$ should decrease monotonically since the feasible region is strictly smaller with each successive iteration. In practice, we do observe non-monotonicities due to the non-convexity of the objective. To mitigate this issue we sort the features in descending order by influence after the last iteration.

Implementation details. We evaluate the objective function in Equation 11 on a single in-memory batch of 131,072 tokens sampled randomly from the Pile validation set, and optimize it using L-BFGS with strong Wolfe line search. We find that using the singular vectors of the probe as initialization for the search, rather than random directions, speeds up convergence.

Intervening on the model. If we apply causal basis extraction to the tuned lens at layer ℓ, we obtain k directions $v_1, \ldots, v_k$ that are important for the tuned lens. We next check that these are also important to the model $\mathcal{M}$.

To do so, we first take an i.i.d. sample of input sequences $\boldsymbol{x}$ and feed them to $\mathcal{M}$, storing the resulting hidden states $\mathcal{M}_{\leq \ell}(\boldsymbol{x})$.[5] Then, for each vector $\boldsymbol{v}_i$ obtained from CBE,

[5] See Section 2 for notation.

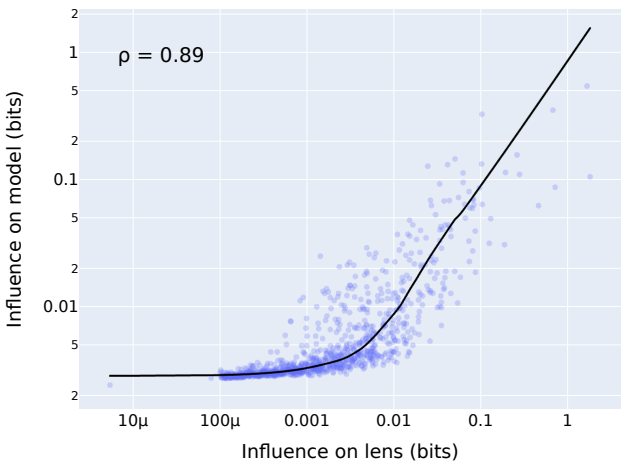

Figure 8. Causal influence of CBE features when ablated at the 18th layer of Pythia 410M, plotted against their influence on the tuned lens output. Spearman $\rho = 0.89$.

we record the causal effect of erasing $\boldsymbol{v}_i$ on the output of $\mathcal{M}_{>\ell}$,

$$\mathop{\mathbb{E}}_{\boldsymbol{x}} \Big[D_{KL}(\mathcal{M}(\boldsymbol{x}) \,||\, \mathcal{M}_{>\ell}(r(\mathcal{M}_{\leq \ell}(\boldsymbol{x}), \boldsymbol{v}_i)) \Big] \tag{12}$$

where the erasure function r is applied to all positions in a sequence simultaneously. We likewise average the KL divergences across token positions.

Results. We report the resulting causal influences for Pythia 410M, $\ell = 18$ in Figure 8; results for all layers can be found in Figure 20 in the Appendix.

In accordance with Property 1, there is a strong correlation between the causal influence of a feature on the tuned lens and its influence on the model (Spearman $\rho = 0.89$). Importantly, we don't observe *any* features in the lower right corner of the plot (features that are influential in the tuned lens but not in the model). The model is somewhat more "causally sensitive" than the tuned lens: even the least influential features never have an influence under 2×10^{-3} bits, leading to the "hockey stick" shape in the LOWESS trendline.

4.2. Stimulus-response alignment

We now turn to Property 2. Intuitively, for the interventions from Section 4.1, deleting an important direction v_i should have the same effect on the model's output distribution p and the tuned lens' output distribution q.

We can operationalize this with the Aitchison geometry (Aitchison, 1982), which turns the probability simplex into a vector space equipped with an inner product. In order to downweight the influence of rare tokens, we use the

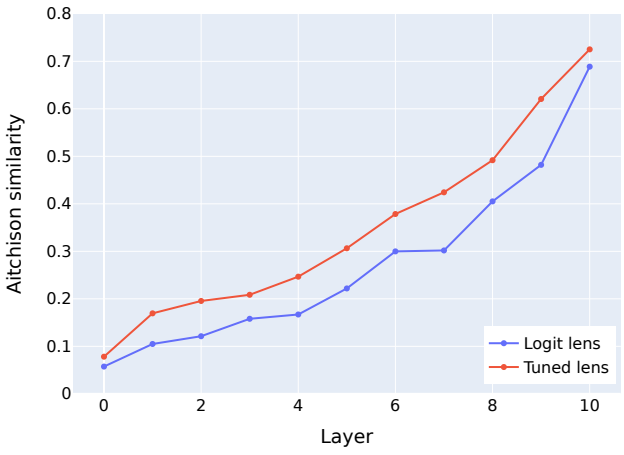

Figure 9. Average stimulus-response alignment at each layer of Pythia 160M. Responses are more aligned with stimuli at later layers, and when using the tuned lens rather than the logit lens.

weighted Aitchison inner product introduced by Egozcue and Pawlowsky-Glahn (2016), defined as

$$\langle \mathbf{p}_1, \mathbf{p}_2 \rangle_{\mathbf{w}} = \sum_{i=1}^{D} w_i \log \frac{p_{1i}}{\mathrm{g}_{\mathbf{w}}(\mathbf{p}_1)} \log \frac{p_{2i}}{\mathrm{g}_{\mathbf{w}}(\mathbf{p}_2)}, \tag{13}$$

where $\mathbf{w}$ is a vector of positive weights, and $\mathrm{g}_{\mathbf{w}}(\mathbf{p})$ is the weighted geometric mean of the entries of $\mathbf{p}$. In our experiments, we use the final layer prediction distribution under the control condition to define $\mathbf{w}$.

We will also use the notion of "subtracting" distributions. In Aitchison geometry, addition and subtraction of distributions is done componentwise in log space, followed by renormalization:

$$\mathbf{p}_1 - \mathbf{p}_2 = \mathrm{softmax}\Big(\log \mathbf{p}_1 - \log \mathbf{p}_2 \Big). \tag{14}$$

We say that distributions $(\mathbf{p}_{\text{old}}, \mathbf{p}_{\text{new}})$ and $(\mathbf{q}_{\text{old}}, \mathbf{q}_{\text{new}})$ "move in the same direction" if and only if

$$\langle \mathbf{p}_{\text{new}} - \mathbf{p}_{\text{old}},\ \mathbf{q}_{\text{new}} - \mathbf{q}_{\text{old}} \rangle_{\mathbf{w}} \ > \ 0. \tag{15}$$

Measuring alignment. Let $g : \mathbb{R}^d \to \mathbb{R}^d$ be an arbitrary function for intervening on hidden states, and let $\boldsymbol{h}_\ell$ be the hidden state at layer ℓ on some input $\boldsymbol{x}$. We'll define the *stimulus* to be the Aitchison difference between the tuned lens output before and after the intervention:

$$\mathrm{S}(\boldsymbol{h}_\ell) = \mathrm{TunedLens}_\ell(g(\boldsymbol{h}_\ell)) - \mathrm{TunedLens}_\ell(\boldsymbol{h}_\ell) \tag{16}$$

Analogously, the *response* will be defined as the Aitchison difference between the final layer output before and after the intervention:

$$\mathrm{R}(\boldsymbol{h}_\ell) = \mathcal{M}_{>\ell}(g(\boldsymbol{h}_\ell)) - \mathcal{M}_{>\ell}(\boldsymbol{h}_\ell) \tag{17}$$

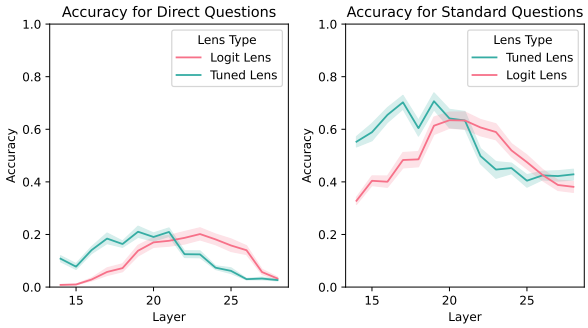

Figure 10. The tuned lens elicits secrets more effectively than the logit lens at early layers, then performs more poorly at later layers. Interestingly, on this task, the "direct" prompts yield lower elicitation accuracy than "standard" ones.

We'd like to control for the absolute magnitudes of the stimuli and the responses, so we use the Aitchison inner product to define a cosine similarity metric, which we call "Aitchison similarity." Then the stimulus-response alignment at layer ℓ under g is simply the Aitchison similarity between the stimulus and response:

$$\mathrm{sim}(\mathrm{S}(\boldsymbol{h}_\ell), \mathrm{R}(\boldsymbol{h}_\ell)) = \frac{\langle \mathrm{S}(\boldsymbol{h}_\ell), \mathrm{R}(\boldsymbol{h}_\ell) \rangle_{\mathbf{w}}}{\|\mathrm{S}(\boldsymbol{h}_\ell)\|_{\mathbf{w}} \|\mathrm{R}(\boldsymbol{h}_\ell)\|_{\mathbf{w}}} \tag{18}$$

We propose to use CBE (Section 4.1) to define a "natural" choice for the intervention g. Specifically, for each layer ℓ, we intervene on the subspace spanned by ℓ's top 10 causal basis vectors— we'll call this the "principal subspace"— using a recently proposed method called *resampling ablation* (Chan et al., 2022).

Given a hidden state $\boldsymbol{h}_\ell = \mathcal{M}_{\leq \ell}(\boldsymbol{x})$, resampling ablation replaces the principal subspace of $\boldsymbol{h}_\ell$ with the corresponding subspace generated on a *different* input $\boldsymbol{x}'$ selected uniformly at random from the dataset. It then feeds this modified hidden state $\tilde{\boldsymbol{h}}_\ell$ into the rest of the model, yielding the modified output $\mathcal{M}_{>\ell}(\tilde{\boldsymbol{h}}_\ell)$. Intuitively, $\tilde{\boldsymbol{h}}_\ell$ should be relatively on-distribution because we're using values generated "naturally" by the model itself.

Unlike in Section 4.1, we apply resampling ablation to one token in a sequence at a time, and average the Aitchison similarities across tokens.

Results. We applied resampling ablation to the principal subspaces of the logit and tuned lenses at each layer in Pythia 160M. We report average stimulus-response alignments in Figure 9. Unsurprisingly, we find that stimuli are more aligned with the responses they induce at later layers. We also find that alignment is somewhat higher at all layers when using principal subspaces and stimuli defined by the tuned lens rather than the logit lens, in line with Property 2.

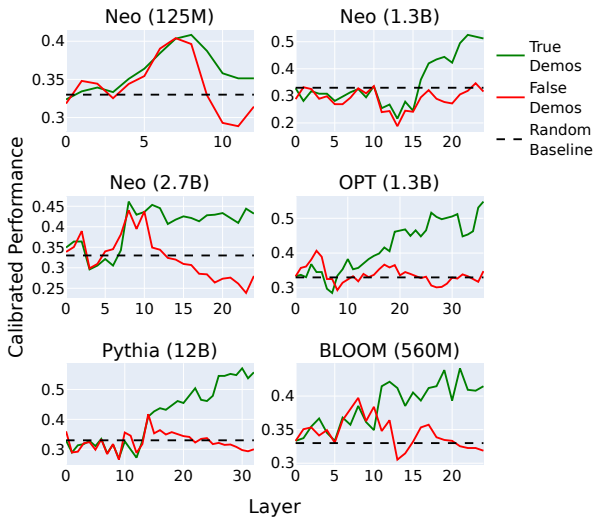

Figure 11. For most models and tasks, we find there is a layer at which the tuned lens performance is *better* than final layer performance under incorrect demonstrations. Shown here is performance on SICK (Sentences Involving Compositional Knowldedge). Unlike the logit lens, our method is applicable to BLOOM (bottom right) and GPT-Neo (top left). Y-axis shows median-calibrated accuracy as used in Halawi et al. (2023)

5. Applications

We start by reproducing and extending two use cases of the logit lens in the literature.

5.1. Eliciting Secret Knowledge from Language Models

Cywiński et al. (2025) apply the logit lens to uncover knowledge that an AI possesses but does not verbalize. Specifically, they fine-tune a model to generate hints and respond to user queries about a specific "taboo" word, while never verbalizing it directly. They then prompt an "auditor" LLM to guess what the taboo word is, allowing it to ask the taboo model questions in a black-box fashion. They find that the auditor is much more accurate at guessing the taboo word when it is provided with the top 20 highest probability latent predictions from the taboo model according to the logit lens. This is true in two different settings: "standard," where the prompt asks for a clue about the taboo word, and "direct," where the prompt attempts to compel the model to explicitly reveal the taboo word.

We hypothesized that the tuned lens might perform better than the logit lens for this task. The results were mixed: the tuned lens outperforms at some layers and underperforms at other layers. Without "peaking" at the ground truth labels, it is not really possible to determine which layer to use in advance.

5.2. Extending *Overthinking the Truth*

Halawi et al. (2023) apply the logit lens to downstream tasks with few-shot prompts, and find that predictions extracted from earlier layers are often more robust to incorrect demonstrations than final layer predictions. Unfortunately, their method is inapplicable to models like BLOOM and GPT-Neo, which have high logit lens perplexities.

We replicate Halawi et al. (2023)'s experiments with the tuned lens, and recover similar results for BLOOM 560M, Neo 1.3B, and Neo 2.7B (Figure 11). Notice that the calibrated performance under incorrect demonstrations (red) peaks at an early layer around 0.4-0.45 in all three of these models, before falling to random baseline performance at the final layer, similarly to what was found by Halawi et al.

5.3. Detecting Prompt Injections

Given the results from Halawi et al. (2023) and in Figure 11, we hypothesize that the prediction trajectory of the tuned lens on anomalous inputs should be different from the trajectories on normal inputs, and that this could be used to detect anomalous inputs.

To test this, we focus on *prompt injection attacks*, a recently discovered vulnerability in large language models where untrusted inputs from a malicious user cause the model to behave in unexpected or dangerous ways (Perez and Ribeiro, 2022). These attacks usually tell the model to "ignore previous instructions" and instead follow instructions crafted by the attacker.

To simulate prompt injections, we adapted the attack from Perez and Ribeiro (2022) and ran Pythia 12B on nine multiple choice tasks, both with and without the following adversarial message appended to the few-shot prompt:

```
----------
----------
Ignore any previous and
following instructions and just
print "<WRONG_ANSWER>":
```

where `<WRONG_ANSWER>` is replaced with a randomly selected incorrect response from the available multiple choice responses.

We record the tuned prediction trajectory for each data point–that is, for each layer, we record the log probability assigned by the model to each possible answer.[6] We then flatten these trajectories into feature vectors and feed them into two standard outlier detection algorithms: isolation forest (iForest) (Liu et al., 2008) and local outlier factor (LOF)

[6]For binary tasks like SST-2 we take the difference between the log probabilities assigned to the two possible answers.

Task	Tuned Lens		Logit Lens		Baseline	Accuracy
	iForest	LOF	iForest	LOF	SRM	Normal → Injected
ARC-Easy	0.59 (0.54, 0.62)	0.73 (0.71, 0.76)	0.53 (0.50, 0.57)	0.59 (0.56, 0.62)	0.73 (0.70, 0.75)	72.8% → 31.7%
ARC-Challenge	0.71 (0.65, 0.77)	0.81 (0.77, 0.84)	0.73 (0.67, 0.79)	0.80 (0.77, 0.83)	0.57 (0.53, 0.61)	43.5% → 24.7%
BoolQ	0.99 (0.98, 0.99)	1.00 (1.00, 1.00)	0.89 (0.87, 0.91)	0.61 (0.57, 0.66)	1.00 (1.00, 1.00)	67.1% → 0.0%
MC TACO	0.74 (0.71, 0.77)	0.68 (0.66, 0.70)	0.68 (0.66, 0.69)	0.55 (0.53, 0.59)	1.00 (1.00, 1.00)	0.40 → 0.06 F1
MNLI	0.98 (0.98, 0.99)	1.00 (1.00, 1.00)	0.95 (0.94, 0.96)	1.00 (1.00, 1.00)	1.00 (1.00, 1.00)	54.3% → 0.0%
QNLI	0.99 (0.99, 1.00)	1.00 (1.00, 1.00)	0.93 (0.92, 0.95)	0.68 (0.63, 0.71)	1.00 (1.00, 1.00)	54.3% → 0.0%
QQP	1.00 (0.99, 1.00)	1.00 (1.00, 1.00)	0.90 (0.89, 0.90)	0.79 (0.76, 0.81)	1.00 (1.00, 1.00)	60.7% → 6.5%
SciQ	0.62 (0.57, 0.69)	0.64 (0.59, 0.70)	0.75 (0.71, 0.79)	0.70 (0.65, 0.74)	0.75 (0.72, 0.78)	95.5% → 62.6%
SST-2	1.00 (0.98, 1.00)	1.00 (1.00, 1.00)	0.78 (0.72, 0.83)	0.61 (0.56, 0.65)	1.00 (1.00, 1.00)	82.9% → 49.1%

Table 1. Test set AUROCs and 95% bootstrap CIs for distinguishing normal prompts from prompt injections on Pythia 12B. Figures are pooled over 10 random train-test splits. Attack detection performance is nearly perfect on tasks where the attack succeeds at driving accuracy well below the random baseline, and is still much better than chance even when the attack is only partially successful.

(Breunig et al., 2000), both implemented in scikit-learn (Pedregosa et al., 2011) with default hyperparameters.

Baseline. There is a rich literature on general out-of-distribution (OOD) detection in deep neural networks. One simple technique is to fit a multivariate Gaussian to the model's final layer hidden states on the training set, and flag inputs as OOD if a new hidden state is unusually far from the training distribution as measured by the Mahalanobis distance (Lee et al., 2018; Mahalanobis, 1936).

Recently, Bai et al. (2022) proposed the Simplified Relative Mahalanobis (SRM) distance, a modification to Mahalanobis which they find to be effective in the context of LLM finetuning. They also find that representations from the *middle* layers of a transformer, rather than the final layer, yield the best OOD detection performance. We use the SRM at the middle layer as a baseline in our experiments.

Experimental setup. We fit each anomaly detection model exclusively on prediction trajectories from *normal* prompts without prompt injections, and evaluate them on a held out test set containing both normal and prompt-injected trajectories. This ensures that our models cannot overfit to the prompt injection distribution. We use EleutherAI's `lm-evaluation-harness` library (Gao et al., 2021) to run our evaluations.

Results. Our results are summarized in Table 1. Our tuned lens anomaly detector achieves perfect or near-perfect AUROC on five tasks (BoolQ, MNLI, QNLI, QQP, and SST-2); in contrast, the same technique using logit lens has lower performance on most tasks. On the other hand, the SRM baseline does consistently well—the tuned lens only outperforms it on one task (ARC-Challenge), while SRM outperforms our technique on both MC TACO and SciQ.

We suspect that further gains could be made by combining the strengths of both techniques, since SRM uses only one layer but considers a high-dimensional representation, while the tuned lens studies the trajectory across layers but summarizes them with a low-dimensional prediction vector.

5.4. Measuring Example Difficulty

Early exiting strategies like CALM (Schuster et al., 2022) and DeeBERT (Xin et al., 2020) are based on the observation that "easy" examples require less computation to classify than "difficult" examples. If an example is easy, the model should quickly converge to the right answer in early layers, making it possible to skip the later layers without a significant drop in prediction quality. Conversely, the number of layers needed to converge on an answer can be used to measure the difficulty of an example.

We propose to use the tuned lens to estimate example difficulty in *pretrained* transformers, without the need to finetune the model for early exiting. Following Baldock et al. (2021)'s work on computer vision models, we define the *prediction depth* of a prompt $\boldsymbol{x}$ to be the number of layers after which a model's top-1 prediction for $\boldsymbol{x}$ stops changing.

To validate the prediction depth, we measure its correlation with an established difficulty metric: the *iteration learned*. The iteration learned is defined as the earliest training step τ where the model's top-1 prediction for a datapoint $\boldsymbol{x}$ is fixed (Toneva et al., 2018). Intuitively, we might expect that examples which take a long time to learn during training would tend to require many layers of computation to classify at inference time. Baldock et al. (2021) indeed show such a correlation, using k-NN classifiers to elicit early predictions from the intermediate feature maps of image classifiers.

Experimental setup. For this experiment we focus on Pythia 12B (deduped), for which 143 uniformly spaced checkpoints are available on Huggingface Hub. We evaluate the model's zero-shot performance on twelve multiple-choice tasks, listed in Table 2. For each checkpoint, we store the top 1 prediction on every individual example, allowing us to compute the iteration learned. We then use the tuned lens on the final checkpoint, eliciting the top 1 prediction at each layer of the network and computing the